THE UNIVERSITY OF AZAD JAMMU AND KASHMIR, MUZAFFARABAD



COURSE TITLE	Digital Image Processing
COURSE CODE	CS-3112
LAB TITLE	Digital Images, Pixels, Color & Basic Transformations
STUDENT NAME	MAHDI ALI NAHVI
STUDENT ROLL NUMBER	2024-SE-16
INSTRUCTOR NAME	Engr. Fahad Nisar
SUBMISSION DATE	October 29, 2026
LAB NUMBER	02

Objective

To use NumPy, OpenCV and Matplotlib to load, inspect, display, analyse, transform and save a digital image, while handling coordinates, colour channels, histograms, resolution and bit depth correctly. All results in this report come from running lab02.py on my own image, Butterfly.jpg (1408 × 768 pixels, colour JPG). Under each step, the code screenshot is followed by the output screenshot and then the figure, if there is one.

Step 1: Project and Virtual Environment

The project folder dip-lab-02 was created, and a virtual environment was made inside it with python -m venv .venv. A virtual environment keeps this lab's libraries separate from the system Python so versions do not conflict between projects. After activating it (the (.venv) prefix appears in the prompt), numpy, opencv-python and matplotlib were installed. The import check prints the path of .venv\Scripts\python.exe, which proves that the interpreter in use is the one inside the virtual environment. Finally .venv/ was added to .gitignore so the environment is never submitted.

1.1 Create the project folder

```
D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02>mkdir dip-lab-02

D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02>ls
 2024-SE-2024-SE-16-LAB02.docx  'Lab Task 02.docx'   README.md   dip-lab-02   lab02.py   lab2.ipynb
```

1.2 Create the virtual environment

```
PS D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02> python -m venv .venv
PS D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>
```

1.3 Activate the environment

```
D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>.venv\Scripts\activate.bat

(.venv) D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>
```

1.4 Install numpy

```
(.venv) D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>pip install numpy
Requirement already satisfied: numpy in .\.venv\Lib\site-packages (2.5.4)
```

1.5 Install opencv-python

```
(.venv) D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>pip install opencv-python
Collecting opencv-python
  Using cached opencv_python-5.0.0.93-cp37-abi3-win_amd64.whl.metadata (20 kB)
Requirement already satisfied: numpy>=2 in .\.venv\Lib\site-packages (from opencv-python) (2.5.4)
Downloading opencv_python-5.0.0.93-cp37-abi3-win_amd64.whl (44.0 MB)
   ---------------------------------------- 44.0/44.0 MB 455.7 kB/s  0:01:26
Installing collected packages: opencv-python
Successfully installed opencv-python-5.0.0.93
```

1.6 Install matplotlib

```
(.venv) D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>pip install matplotlib
Collecting matplotlib
  Using cached matplotlib-3.11.2-cp312-cp312-win_amd64.whl.metadata (80 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Using cached contourpy-1.4.0-cp312-cp312-win_amd64.whl.metadata (3.8 kB)
Collecting cycler>=0.10 (from matplotlib)
  Using cached cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.28.2 (from matplotlib)
  Downloading fonttools-4.68.0-cp312-cp312-win_amd64.whl.metadata (142 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Using cached kiwisolver-1.5.1-cp312-cp312-win_amd64.whl.metadata (5.2 kB)
Requirement already satisfied: numpy>=1.25 in .\.venv\Lib\site-packages (from matplotlib) (2.5.4)
Collecting packaging>=20.0 (from matplotlib)
  Downloading packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pillow>=9 (from matplotlib)
  Using cached pillow-13.0.0-cp312-cp312-win_amd64.whl.metadata (9.3 kB)
Collecting pyparsing>=3 (from matplotlib)
  Using cached pyparsing-3.3.3-py3-none-any.whl.metadata (5.9 kB)
Collecting python-dateutil>=2.7 (from matplotlib)
  Downloading python_dateutil-3.0.0-py2.py3-none-any.whl.metadata (8.2 kB)
Collecting six>=1.6 (from python-dateutil>=2.7->matplotlib)
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Collecting tzdata (from python-dateutil>=2.7->matplotlib)
  Downloading tzdata-2026.6-py2.py3-none-any.whl.metadata (1.4 kB)
Using cached matplotlib-3.11.2-cp312-cp312-win_amd64.whl (9.3 MB)
Using cached contourpy-1.4.0-cp312-cp312-win_amd64.whl (234 kB)
Using cached cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.68.0-cp312-cp312-win_amd64.whl (2.5 MB)
   ---------------------------------------- 2.5/2.5 MB 649.0 kB/s  0:00:03
Using cached kiwisolver-1.5.1-cp312-cp312-win_amd64.whl (70 kB)
Downloading packaging-26.3-py3-none-any.whl (129 kB)
Using cached pillow-13.0.0-cp312-cp312-win_amd64.whl (7.3 MB)
```

```
Using cached pyparsing-3.3.3-py3-none-any.whl (126 kB)
Downloading python_dateutil-3.0.0-py2.py3-none-any.whl (94 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading tzdata-2026.6-py2.py3-none-any.whl (348 kB)
Installing collected packages: tzdata, six, pyparsing, pillow, packaging, kiwisolver, fonttools, cycler, contourpy, python-dateutil,
matplotlib
Successfully installed contourpy-1.4.0 cycler-0.12.1 fonttools-4.68.0 kiwisolver-1.5.1 matplotlib-3.11.2 packaging-26.3 pillow-13.0.0
 pyparsing-3.3.3 python-dateutil-3.0.0 six-1.17.0 tzdata-2026.6
```

1.7 Verify imports and executable, add .venv/ to .gitignore

```
(.venv) D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>python -c "import sys, numpy, cv2, matplot
lib; print(sys.executable)"
D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02\.venv\Scripts\python.exe

(.venv) D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>echo .venv/ >> .gitignore
```

Evidence: the installed versions are numpy 2.5.4, opencv-python 5.0.0.93 and matplotlib 3.11.2, and the Python executable is located inside the project's .venv folder.

Step 2: Load the Image

The libraries are imported with the standard aliases (cv, np, plt). cv.imread() does not raise an error when the file is missing; it silently returns None. The script therefore checks for None and raises FileNotFoundError, so a wrong file name or path is caught immediately instead of causing a confusing error later. The image is stored as a NumPy array of shape (768, 1408, 3): 768 rows, 1408 columns and 3 colour channels, with data type uint8, so every value is an integer from 0 to 255. (The Matplotlib font-cache message appears only the first time Matplotlib runs.)

Code

lab02.py

```python
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

IMAGE_PATH = "Butterfly.jpg"
image = cv.imread(IMAGE_PATH)                    # OpenCV loads in BGR order
if image is None:
    raise FileNotFoundError(f"Image not found or unreadable: {IMAGE_PATH}")
print("Image shape (H, W, C):", image.shape)
print("Image data type      :", image.dtype)
```

Output

```
(.venv) D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>python lab02.py
Matplotlib is building the font cache; this may take a moment.
Image shape (H, W, C): (768, 1408, 3)
Image data type      : uint8
```

Step 3: Inspect the Image

NumPy arrays are indexed as [row, column], that is [y, x], with the origin (0, 0) at the top-left corner. The centre pixel is therefore at [height // 2, width // 2] = [384, 704]. OpenCV stores channels in B, G, R order, so every pixel is printed as (B, G, R). All printed results are labelled.

Code

lab02.py

```python
height, width, channels = image.shape
print("Height (rows)        :", height)
print("Width (columns)      :", width)
print("Number of channels   :", channels)

center_row, center_col = height // 2, width // 2
print("Top-left pixel [0, 0] (B, G, R)          :", image[0, 0])
print(f"Center pixel [{center_row}, {center_col}] (B, G, R)      :", image[center_row, center_col])

print("Global minimum value :", image.min())
print("Global maximum value :", image.max())
for i, name in enumerate(["Blue", "Green", "Red"]):          # BGR order
    print(f"{name:5s} channel -> min: {image[:, :, i].min():3d}, max: {image[:, :, i].max():3d}")
```

Output

```
(.venv) D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>python lab02.py
Image shape (H, W, C): (768, 1408, 3)
Image data type      : uint8
Height (rows)        : 768
Width (columns)      : 1408
Number of channels   : 3
Top-left pixel [0, 0] (B, G, R)          : [174 203 218]
Center pixel [384, 704] (B, G, R)      : [25 20 21]
Global minimum value : 0
Global maximum value : 255
Blue  channel -> min:   0, max: 255
Green channel -> min:   0, max: 255
Red   channel -> min:   0, max: 255
```

- **Dimensions:** height 768, width 1408, 3 channels.
- **Top-left pixel [0, 0]:** B = 174, G = 203, R = 218, i.e. RGB (218, 203, 174), a pale cream colour. This matches the soft, light-yellow blurred background in the top-left corner of the picture.
- **Centre pixel [384, 704]:** B = 25, G = 20, R = 21, i.e. almost black. The centre of the picture falls on the dark body of the butterfly.
- **Ranges:** the global minimum is 0 and the maximum is 255, and each of the blue, green and red channels also spans 0–255. The image therefore uses the full 8-bit range, from pure black to pure white.

Step 4: Colour Conversion and Channel Separation

OpenCV loads colour images as BGR, but Matplotlib expects RGB, so the image was converted with cv.cvtColor(image, cv.COLOR_BGR2RGB). A grayscale copy was created with COLOR_BGR2GRAY, and the red, green and blue channels were separated by slicing the last axis (rgb[:, :, 0], [:, :, 1], [:, :, 2]). The individual channels are single-channel (2-D) arrays, so they are displayed with cmap="gray"; a brighter area means a larger amount of that colour. A 2 × 3 figure with titles and hidden axes shows all six views.

Code

lab02.py

```python
rgb = cv.cvtColor(image, cv.COLOR_BGR2RGB)
gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
red, green, blue = rgb[:, :, 0], rgb[:, :, 1], rgb[:, :, 2]
print("RGB shape:", rgb.shape, "| Grayscale shape:", gray.shape)

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
panels = [(rgb, "RGB (correct colours)", None),
          (red, "Red channel", "gray"),
          (green, "Green channel", "gray"),
          (blue, "Blue channel", "gray"),
          (gray, "Grayscale", "gray"),
          (image, "BGR shown without conversion", None)]
for ax, (img, title, cmap) in zip(axes.ravel(), panels):
    ax.imshow(img, cmap=cmap)
    ax.set_title(title)
    ax.axis("off")
plt.tight_layout()
plt.savefig("channels_figure.png", dpi=110)
plt.show()

r, g, b = (int(v) for v in rgb[center_row, center_col])
hex_value = "#{:02X}{:02X}{:02X}".format(r, g, b)
print(f"Center pixel RGB = ({r}, {g}, {b})  ->  HEX = {hex_value}")
```

Output and figure (2 × 3: RGB, red, green, blue, grayscale, unconverted BGR)



Observations: In the correct RGB view the wings are orange, the flowers are lavender and the background is a soft green-yellow. The red channel is very bright on the wings because orange contains a lot of red. The blue channel is very dark on the wings because orange contains almost no blue, while the purple flowers are bright in it. The green channel lies in between. The grayscale image is a weighted mix of the three channels. In the unconverted BGR view, Matplotlib treats the blue channel as red and the red channel as blue, so the orange wings appear bright blue and the background turns teal and pink. This shows why the BGR-to-RGB conversion is needed before display.

HEX value of the centre pixel: the centre pixel is BGR [25 20 21] (Step 3), which in RGB order is (21, 20, 25). Converting each value to two hexadecimal digits gives 21 = 0x15, 20 = 0x14 and 25 = 0x19, so the HEX colour is #151419 (a very dark, near-black colour). The code prints it with the format "#{:02X}{:02X}{:02X}".

Step 5: Crop, Resize and Save

Cropping is plain NumPy slicing, image[y_start:y_end, x_start:x_end], with rows first. The slice [250:350, 510:610] selects exactly 100 rows and 100 columns, because the end index is excluded; an assert statement confirms the size. The crop lies on an orange wing, so the veins are visible. cv.resize() takes the size as (width, height) and was used to enlarge the crop to 200 × 200 with bilinear interpolation. The resized RGB image was converted back to BGR before cv.imwrite(), because OpenCV writes files assuming BGR order; otherwise the saved colours would be wrong. cv.imwrite() returned True, so resized_crop.png was saved successfully.

Code

lab02.py

```python
y_start, y_end = 250, 350        # 100 rows
x_start, x_end = 510, 610        # 100 columns (region around the sun)
crop = rgb[y_start:y_end, x_start:x_end]
print("Crop shape (H, W, C)   :", crop.shape)
assert crop.shape[:2] == (100, 100), "Crop must be exactly 100 x 100"

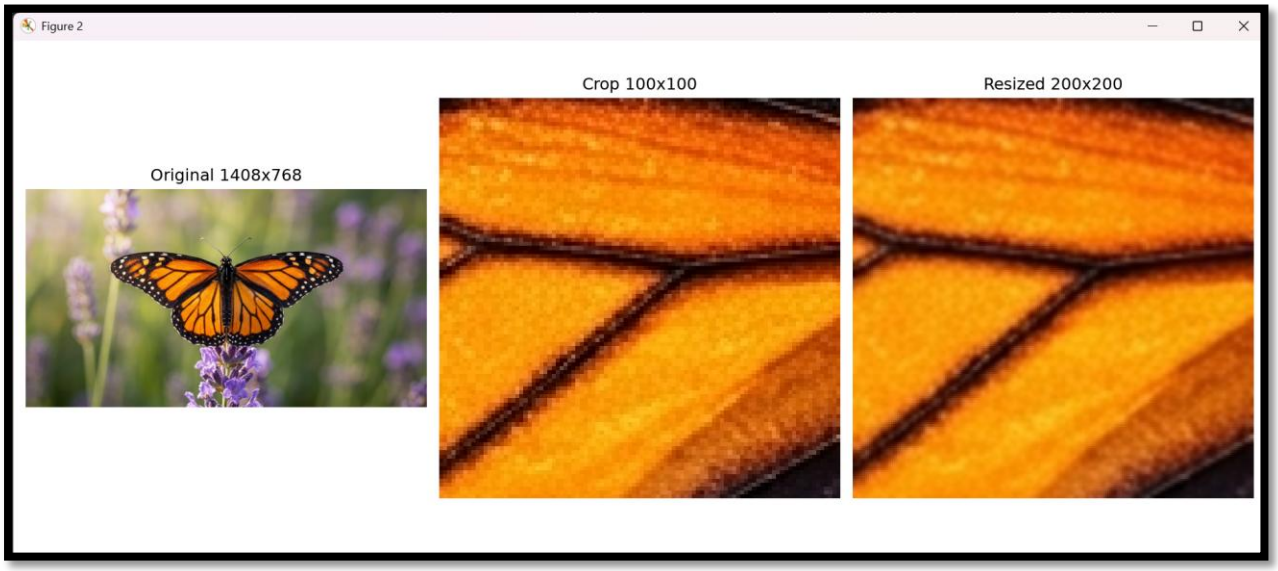
resized = cv.resize(crop, (200, 200), interpolation=cv.INTER_LINEAR)   # (width, height)
print("Resized shape (H, W, C):", resized.shape)

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(rgb);     axes[0].set_title(f"Original {rgb.shape[1]}x{rgb.shape[0]}")
axes[1].imshow(crop);    axes[1].set_title("Crop 100x100")
axes[2].imshow(resized); axes[2].set_title("Resized 200x200")
for ax in axes:
    ax.axis("off")
plt.tight_layout()
plt.savefig("crop_resize_figure.png", dpi=110)
plt.show()
resized_bgr = cv.cvtColor(resized, cv.COLOR_RGB2BGR)       # imwrite expects BGR
saved = cv.imwrite("resized_crop.png", resized_bgr)
print("cv.imwrite() saved resized_crop.png successfully:", saved)
```

Output

```
(.venv) D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>python lab02.py
Crop shape (H, W, C)   : (100, 100, 3)
Resized shape (H, W, C): (200, 200, 3)
cv.imwrite() saved resized_crop.png successfully: True
```

Figure (original image, 100 × 100 crop, 200 × 200 resized crop)



In the crop, individual square pixels are visible. In the resized version the same content is twice as large and looks smoother, but not sharper: interpolation estimates the new pixels from their neighbours and cannot add real detail.

Step 6: Grayscale Histogram

The 2-D grayscale image was flattened into a 1-D array of 1,081,344 pixel values (768 × 1408) and plotted with 256 bins over the range 0–256, so that each bin matches exactly one intensity level from 0 to 255. The title, axis labels and the saved file grayscale_histogram.png are included. The mean, median and the share of dark (< 85), middle (85–170) and bright (> 170) pixels were also printed so that the interpretation is based on numbers.

Code

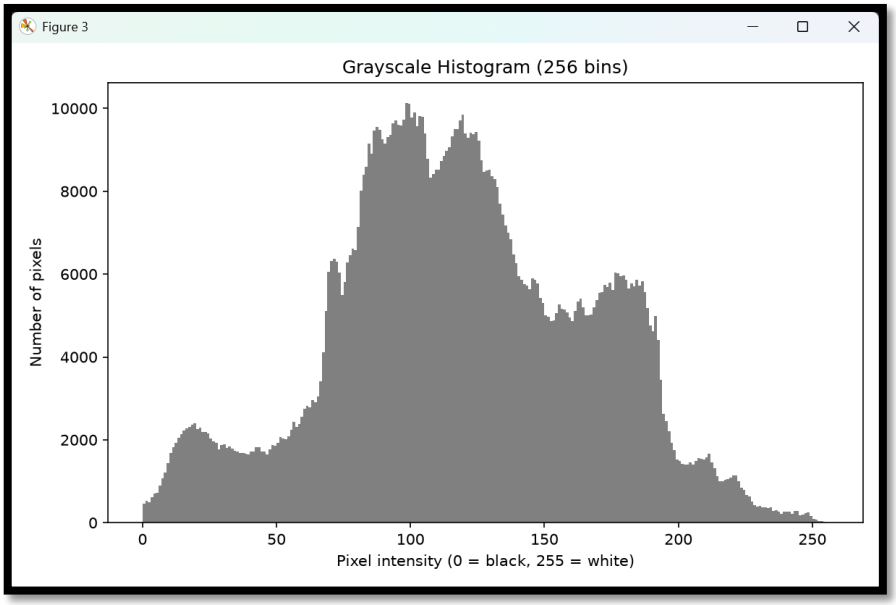
lab02.py

```
73  pixels = gray.flatten()
74  plt.figure(figsize=(8, 5))
75  plt.hist(pixels, bins=256, range=(0, 256), color="gray")
76  plt.title("Grayscale Histogram (256 bins)")
77  plt.xlabel("Pixel intensity (0 = black, 255 = white)")
78  plt.ylabel("Number of pixels")
79  plt.tight_layout()
80  plt.savefig("grayscale_histogram.png", dpi=110)
81  plt.show()
82
83  print("Mean intensity      :", round(float(pixels.mean()), 2))
84  print("Median intensity    :", float(np.median(pixels)))
85  print("Pixels < 85 (dark)  : {:.1f} %".format(100 * np.mean(pixels < 85)))
86  print("Pixels 85-170 (mid) : {:.1f} %".format(100 * np.mean((pixels >= 85) & (pixels <= 170))))
87  print("Pixels > 170 (light): {:.1f} %".format(100 * np.mean(pixels > 170)))
```

Output

```
(.venv) D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>python lab02.py
Mean intensity      : 118.16
Median intensity    : 116.0
Pixels < 85 (dark)  : 22.8 %
Pixels 85-170 (mid) : 60.7 %
Pixels > 170 (light): 16.5 %
```

Figure (256-bin grayscale histogram)



Interpretation: The image is mainly balanced. The mean intensity is 118.16 and the median is 116, close to the middle of the range (127), and 60.7% of all pixels are mid-tones, compared with 22.8% dark and 16.5% bright pixels. The histogram is a wide hill between about 70 and 140 with a second, smaller hump near 180, and a small hump near 20 from the dark butterfly body and wing borders. It uses nearly the whole range, which means the image has good contrast and is neither mostly dark nor mostly bright.

Step 7: Bit Depth

An 8-bit horizontal gradient (100 × 256 pixels, values 0 to 255) was created with np.tile. It was then quantized to 1, 2 and 4 bits, which allow 2, 4 and 16 intensity levels. Each pixel is mapped to its level index (value // step) and then scaled back to 0–255 so that the images can be compared on the same display range. The printed unique-value counts are 256, 2, 4 and 16, which equal 2^8, 2^1, 2^2 and 2^4.

Code

lab02.py

```python
gradient = np.tile(np.arange(256, dtype=np.uint8), (100, 1))       # 100 x 256, values 0..255

def quantize(img, bits):
    levels = 2 ** bits
    step = 256 // levels
    q = (img // step)                                    # level index 0 .. levels-1
    return (q * (255 // (levels - 1))).astype(np.uint8)  # stretch back to 0..255 for display

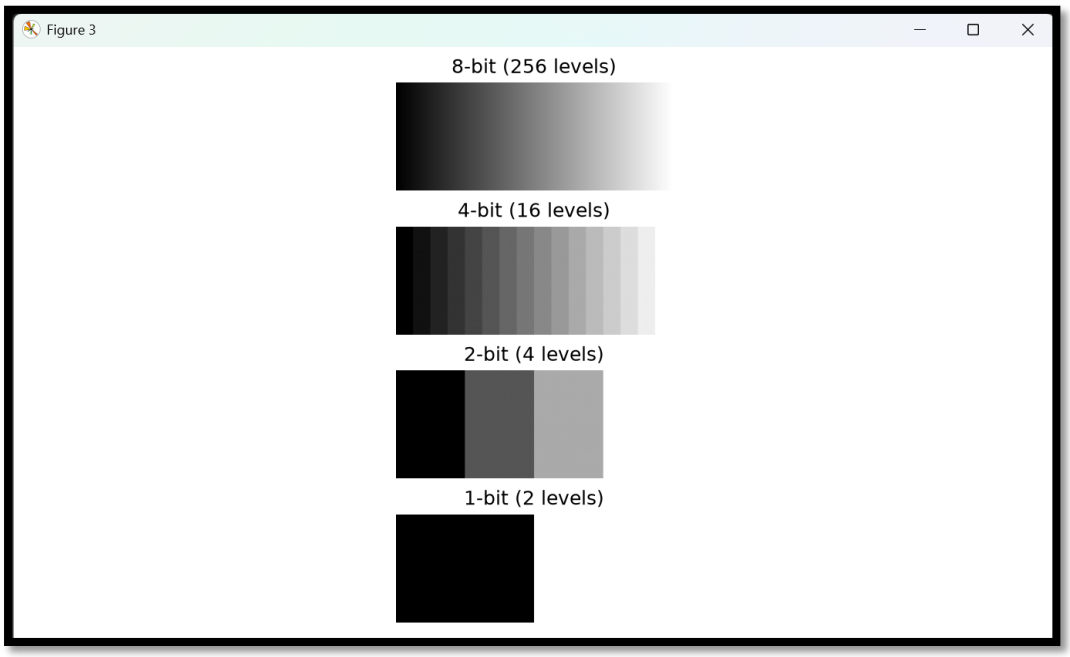
g1, g2, g4 = quantize(gradient, 1), quantize(gradient, 2), quantize(gradient, 4)
fig, axes = plt.subplots(4, 1, figsize=(9, 6))
for ax, (img, title) in zip(axes, [(gradient, "8-bit (256 levels)"), (g4, "4-bit (16 levels)"),
                                   (g2, "2-bit (4 levels)"), (g1, "1-bit (2 levels)")]):
    ax.imshow(img, cmap="gray", vmin=0, vmax=255)
    ax.set_title(title)
    ax.axis("off")
plt.tight_layout()
plt.savefig("bit_depth_figure.png", dpi=110)
plt.show()

print("Unique values in 8-bit gradient:", len(np.unique(gradient)))
print("Unique values in 1-bit gradient:", len(np.unique(g1)))
print("Unique values in 2-bit gradient:", len(np.unique(g2)))
print("Unique values in 4-bit gradient:", len(np.unique(g4)))
```

Output

```
(.venv) D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>python lab02.py
Unique values in 8-bit gradient: 256
Unique values in 1-bit gradient: 2
Unique values in 2-bit gradient: 4
Unique values in 4-bit gradient: 16
```

Figure (original 8-bit and 4-, 2- and 1-bit gradients)



With fewer bits the smooth gradient breaks into visible bands (false contours). The 4-bit version still looks fairly smooth, the 2-bit version shows distinct bands, and the 1-bit version has only black and white. The brightest band of each image is white (255) and is therefore not visible against the white figure background. This is why the 2-bit image looks like three bands and the 1-bit image looks like a single black block, even though they really contain 4 levels (0, 85, 170, 255) and 2 levels (0, 255).

Step 8: Final Run

lab02.py was run from the activated environment with python lab02.py, and all results above come from this error-free run. The environment was then closed with deactivate; the (.venv) prefix disappears from the prompt in the screenshot below. The files for submission are lab02.py, Butterfly.jpg, resized_crop.png, grayscale_histogram.png, .gitignore, README.md and this report; the .venv folder is not submitted.

Deactivate the environment

```
(.venv) D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>deactivate
D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 02\dip-lab-02>
```

Explanations in My Own Words

BGR versus RGB

Both describe a colour pixel with blue, green and red intensities; they only differ in the order in which the three values are stored. OpenCV reads images as BGR (blue first), while Matplotlib and most other libraries assume RGB (red first). If the order is not converted, red and blue are swapped; in this lab the orange wings turned blue. So the image is converted with cv.cvtColor(..., COLOR_BGR2RGB) before plt.imshow(), and converted back to BGR before saving with cv.imwrite().

Resolution versus bit depth

Resolution is the number of pixels in an image (here 1408 × 768), which decides how finely the spatial detail is sampled. Bit depth is the number of bits used to store each pixel's value, which decides how many intensity or colour levels are possible (8 bits = 256 levels, 4 bits = 16, 1 bit = 2). A high-resolution image with low bit depth is sharp but shows banding, while a high bit depth with low resolution has smooth tones but blocky detail. Step 5 changed the resolution (100 × 100 to 200 × 200); Step 7 changed the bit depth.

What a histogram shows and does not show

- **Shows:** how many pixels have each intensity value, i.e. the brightness distribution. From it we can tell whether an image is dark, bright or balanced, how much contrast it has (spread of the values), and whether values are clipped at 0 or 255.
- **Does not show:** where the pixels are located (there is no spatial information), what the objects in the image are, or the colour, because a grayscale histogram ignores hue. Two completely different images can have the same histogram, for example if the pixels of one image are shuffled randomly.